

the page, and then send our carrier (the ‘transmission’ object) to the script which will then work its D3 magic and bring about beautiful visualisation for the rather drab numbers.

Then, we have a simple worker function, `loadStatsPage`, which, when invoked, opens a new instance of the page in a new tab in the browser. Super simple stuff. Note that this worker doesn’t actually do anything to open the page, except supply the URL that we want to access. When the listener for the address bar sees this, it automatically does the rest and prepares the page as needed.

Then, there is the preparatory function that takes care of populating the new page with the section that we ‘d like and are working towards creating. It assumes that the page that it is working on is already a blank page that is on the target URL.

Here is the listener that we were talking about, which examines every input in the address bar, to see if we need to load our page there:

```

• <code>
•
• tabs.on('ready', statsOnDemand);
•
• </code>

```

Finally, we’d like to have close control over the workings of the page. For this, we’d need to take a focussed look at all the errors, if any, the code generates. If not, we need to examine that condition by witnessing a statement of success. For this purpose, we create a try-catch block and place the requisite code here.

```

• <code>
•
• try {
•
•   require('about').add({what: 'stats', url: data.url("stats.html")});
•   toolbarButton.moveTo({
•     toolbarID: "nav-bar",
•     forceMove: true
•   });
• }

```